Numpy-Numerical python

used for working with arrays

first we start of with basic array methods and all

Basic

What is Numpy ?

NumPy is a Python library used for working with arrays.

It also has functions for working in domain of linear algebra, fourier transform, and matrices.

NumPy was created in 2005 by Travis Oliphant. It is an open source project and you can use it freely.

NumPy stands for Numerical Python.

Why use Numpy ?

In Python we have lists that serve the purpose of arrays, but they are slow to process.

NumPy aims to provide an array object that is up to 50x faster than traditional Python lists.

The array object in NumPy is called ndarray, it provides a lot of supporting functions that make working with ndarray very easy.

Arrays are very frequently used in data science, where speed and resources are very important.

Why is Numpy faster than list ?

NumPy arrays are stored at one continuous place in memory unlike lists, so processes can access and manipulate them very efficiently.

This behavior is called locality of reference in computer science.

This is the main reason why NumPy is faster than lists. Also it is optimized to work with latest CPU architectures.

Starting off with Numpy

In [1]:
```
import numpy as np
arr=np.array([1,3,5])
# print 1D array
print(arr)
```

```
[1 3 5]
```

In [5]:
```
arr2=np.array([[1,2,3],[4,5,6]])
# print 2D array
print("2D array\n",arr2)
```

```
2D array
 [[1 2 3]
 [4 5 6]]
```

In [6]:
```
#three-dimensional numpy array
a = np.array([[[1, 2, 3], [4, 5, 6]], [[1, 2, 3], [4, 5, 6]]])
print("\n\nthree dimensional\n\n",a)
```

```
three dimensional

 [[[1 2 3]
  [4 5 6]]

 [[1 2 3]
  [4 5 6]]]
```

we can check number of dimension of each array using **ndim**

In [9]:
```
print("dimension of our 1D array : ",arr.ndim)
print("dimension of our 2D array : ",arr2.ndim)
print("dimension of our 3D array : ",a.ndim)
```

```
dimension of our 1D array :  1
dimension of our 2D array :  2
dimension of our 3D array :  3
```

similarly we can also use **ndim** create array of any dimension :

In [11]:
```
import numpy as np

arr3 = np.array([1, 2, 3, 4], ndmin=5)

print(arr3)
print('number of dimensions :', arr3.ndim)
```

```
[[[[[1 2 3 4]]]]]
number of dimensions : 5
```

now before moving any further we try few things :

In [12]:
```
# converting list to numpy array
l1=[[22,25,26,27],[30,25,26,40],[100,1,9,5]]
print("original list : ",l1)
arr4=np.array(l1)
print("list coverted to 2D array : ",arr4)
```

```
original list :  [[22, 25, 26, 27], [30, 25, 26, 40], [100, 1, 9, 5]]
list coverted to 2D array :  [[ 22  25  26  27]
 [ 30  25  26  40]
 [100   1   9   5]]
```

In [14]:
```
# appending values to above array
arr4=np.append(arr4,[[50,42,44,41]])
print("appended : ",arr4)
```

```
appended :  [ 22  25  26  27  30  25  26  40 100   1   9   5  50  42  44  41]
```

In [15]:
```
# finding common between two numpy arrays
b=np.array([100,2,5,3])
c=np.array([100,101,103,2])
print(np.intersect1d(b,c))
```

```
[  2 100]
```

creating array of zeros using **numpy.zeros** method :

a1=np.zeros(1,dtype=int) a2=np.zeros([2,2],dtype=int) a3=np.zeros([3,3],dtype=int) print("1x1 : ",a1) print("2x2 : ",a2) print("3x3 : ",a3)

similary we can create array of ones using **numpy.ones** method

In [17]:
```
a4=np.ones(1,dtype=int)
a5=np.ones([2,2],dtype=int)
a6=np.ones([3,3],dtype=int)
print("1x1 : ",a4)
print("2x2 : ",a5)
print("3x3 : ",a6)
```

```
1x1 :  [1]
2x2 :  [[1 1]
 [1 1]]
3x3 :  [[1 1 1]
 [1 1 1]
 [1 1 1]]
```

we can create identity matrix using **numpy.identity** method

In [20]:
```
# to create a 3x3 identity matrix
import numpy as np
array_2D=np.identity(3)
print('3x3 matrix:')
print(array_2D)
```

```
3x3 matrix:
[[1. 0. 0.]
 [0. 1. 0.]
 [0. 0. 1.]]
```

creating identity matrix with diagonals elements 1 and all other are zero

In [21]:
```
import numpy as np
x = np.eye(3)
print(x)
y = np.eye(3,4)
print(y)
```

```
[[1. 0. 0.]
 [0. 1. 0.]
 [0. 0. 1.]]
[[1. 0. 0. 0.]
 [0. 1. 0. 0.]
 [0. 0. 1. 0.]]
```

to create a 5x5 zero matrix with elements on the main diagonal equal to 1, 2,3,4,5 using **numpy.diagonal** method

In [22]:
```
import numpy as np
x = np.diag([1, 2, 3, 4, 5])
print(x)
```

```
[[1 0 0 0 0]
 [0 2 0 0 0]
 [0 0 3 0 0]
 [0 0 0 4 0]
 [0 0 0 0 5]]
```

check if any value is nan (null) in numpy array using **np.isnan()**

In [23]:
```
a7 = np.array([1, 0, np.nan, 3])
print("Original array")
print(a7)
print("Test element-wise for NaN:")
print(np.isnan(a7))
```

```
Original array
[ 1.  0. nan  3.]
Test element-wise for NaN:
[False False  True False]
```

we can also create numpy arrays smartly by usinh **numpy.arange** method

In [24]:
```
# "arange of numpy array" is similar to "range of list"
# np.arange(starting,ending,step)
print(np.arange(3))
print(np.arange(3.0))
print(np.arange(3,7))
print(np.arange(3,7,2))
print(np.arange(1,4,2))
```

```
[0 1 2]
[0. 1. 2.]
[3 4 5 6]
[3 5]
[1 3]
```

In [25]:
```
#how to create a multi-dimensional array using arange
#it will create array having 3 rows and 4 columns with elements from 10 to 21
a8 = np.arange(10,22).reshape((3, 4))
print(a8)
```

```
[[10 11 12 13]
 [14 15 16 17]
 [18 19 20 21]]
```

Access Numpy array elements

we use indexing,by accessing an array element referring to its index number

In [26]:
```
arr9=np.array([1,2,5,70])
print(arr9[2])# 5 will be printed
```

```
5
```

In [29]:
```
# get first and 4th element and add them
print(arr9[0]+arr9[3])
```

```
71
```

In [30]:
```
arr10 = np.array([[1,2,3,4,5], [6,7,8,9,10]])

print('2nd element on 1st row: ', arr10[0, 1])
```

```
2nd element on 1st row:  2
```

In [31]:
```
arr11 = np.array([[[1, 2, 3], [4, 5, 6]], [[7, 8, 9], [10, 11, 12]]])
# pehli 2D array ki 2nd row ka third element ,6
print(arr11[0, 1, 2])
```

```
6
```

In [32]:
```
arr12 = np.array([[1,2,3,4,5], [6,7,8,9,10]])

print('Last element from 2nd dim: ', arr12[1, -1])
```

```
Last element from 2nd dim:  10
```

Slicing Numpy arrays

In [33]:
```
arr13 = np.array([1, 2, 3, 4, 5, 6, 7])

print(arr13[1:5])# index 1 se index 5 excluding index 5
```

```
[2 3 4 5]
```

In [34]:
```
print(arr13[4:])
```

```
[5 6 7]
```

In [35]:
```
print(arr13[:4])
```

```
[1 2 3 4]
```

In [36]:
```
print(arr13[-3:-1])# here -1 shows 7,-2 is for 6,-3 is for 5,so exclude -1
```

```
[5 6]
```

In [37]:
```
print(arr13[1:5:2])# here 2 shows step,print from index 1 to index 5,not including index 5 ,with gap o
```

```
[2 4]
```

slicing 2D arrays

In [38]:
```
arr14 = np.array([[1, 2, 3, 4, 5], [6, 7, 8, 9, 10]])
#From the second element, slice elements from index 1 to index 4 (not included):
print(arr14[1, 1:4])
```

```
[7 8 9]
```

In [39]:
```
print(arr14[0:2, 2])#From both elements, return index 2:
```

```
[3 8]
```

In [40]:
```
print(arr14[0:2, 1:4])#From both elements, slice index 1 to index 4 (not included), this will return a 2
```

```
[[2 3 4]
 [7 8 9]]
```

Numpy Datatypes

In [41]:
```
arr15 = np.array(['apple', 'banana', 'cherry'])

print(arr15.dtype)
```

```
<U6
```

In [42]:
```
arr16 = np.array([1, 2, 3, 4])
print(arr16.dtype)
```

```
int32
```

In [43]:
```
arr17 = np.array([1, 2, 3, 4], dtype='S')

print(arr17)
print(arr17.dtype)
```

```
[b'1' b'2' b'3' b'4']
|S1
```

Numpy shape,reshape

shape of an array is the number of elements in the array

In [44]:
```
arr18 = np.array([[1, 2, 3, 4], [5, 6, 7, 8]])

print(arr18.shape)
```

```
(2, 4)
```

reshape means changing shape of an array

In [45]:
```
arr19 = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12])

newarr = arr19.reshape(4, 3)

print(newarr)
```

```
[[ 1  2  3]
 [ 4  5  6]
 [ 7  8  9]
 [10 11 12]]
```

In [48]:
```
newarr2 = arr19.reshape(2, 3, 2)

print(newarr2)
```

```
[[[ 1  2]
  [ 3  4]
  [ 5  6]]

 [[ 7  8]
  [ 9 10]
  [11 12]]]
```

iterating arrays

In [50]:
```
for x in arr19:
  print(x,end=" ")
```

```
1 2 3 4 5 6 7 8 9 10 11 12
```

In [52]:
```
for i in arr18:
  for j in i:
    print("{}:{}".format(i,j))
```

```
[1 2 3 4]:1
[1 2 3 4]:2
[1 2 3 4]:3
[1 2 3 4]:4
[5 6 7 8]:5
[5 6 7 8]:6
[5 6 7 8]:7
[5 6 7 8]:8
```

In [53]:
```
for i in arr18:
  print(i,end=" ")
```

```
[1 2 3 4] [5 6 7 8]
```

using **nditer**

In [54]:
```
arr20 = np.array([[[1, 2], [3, 4]], [[5, 6], [7, 8]]])

for x in np.nditer(arr20):
  print(x)
```

```
1
2
3
4
5
6
7
8
```

Array join,split

joining means putting contents of two or more arrays in a single array.

In [55]:
```
import numpy as np

j1 = np.array([1, 2, 3])

j2 = np.array([4, 5, 6])

j3 = np.concatenate((j1, j2))

print(j3)
```

```
[1 2 3 4 5 6]
```

In [56]:
```
# Join two 2-D arrays along rows (axis=1):
import numpy as np

j4 = np.array([[1, 2], [3, 4]])

j5 = np.array([[5, 6], [7, 8]])

j6 = np.concatenate((j4, j5), axis=1)

print(j6)
```

```
[[1 2 5 6]
 [3 4 7 8]]
```

In [57]:
```
j#oining using stacking
import numpy as np

arr1 = np.array([1, 2, 3])

arr2 = np.array([4, 5, 6])

arr = np.stack((arr1, arr2), axis=1)

print(arr)
```

```
[[1 4]
 [2 5]
 [3 6]]
```

In [58]:
```
# using hstack for rows
import numpy as np

arr1 = np.array([1, 2, 3])

arr2 = np.array([4, 5, 6])

arr = np.hstack((arr1, arr2))

print(arr)
```

```
[1 2 3 4 5 6]
```

In [59]:
```
#using vstack for columns
import numpy as np

arr1 = np.array([1, 2, 3])

arr2 = np.array([4, 5, 6])

arr = np.vstack((arr1, arr2))

print(arr)
```

```
[[1 2 3]
 [4 5 6]]
```

spliting array using **array_spliting**

In [60]:
```
import numpy as np

arr = np.array([1, 2, 3, 4, 5, 6])

newarr = np.array_split(arr, 3)

print(newarr)
```

```
[array([1, 2]), array([3, 4]), array([5, 6])]
```

In [61]:
```
import numpy as np

arr = np.array([[1, 2], [3, 4], [5, 6], [7, 8], [9, 10], [11, 12]])

newarr = np.array_split(arr, 3)

print(newarr)
```

```
[array([[1, 2],
       [3, 4]]), array([[5, 6],
       [7, 8]]), array([[ 9, 10],
       [11, 12]])]
```

Array search,sort

ou can search an array for a certain value, and return the indexes that get a match.

To search an array, use the **where()** method.

In [62]:
```
#Find the indexes where the value is 4:


import numpy as np

arr = np.array([1, 2, 3, 4, 5, 4, 4])

x = np.where(arr == 4)

print(x)
```

```
(array([3, 5, 6], dtype=int64),)
```

In [63]:
```
import numpy as np

arr = np.array([1, 2, 3, 4, 5, 6, 7, 8])

x = np.where(arr%2 == 0)

print(x)
```

```
(array([1, 3, 5, 7], dtype=int64),)
```

Sorting means putting elements in an ordered sequence.

Ordered sequence is any sequence that has an order corresponding to elements, like numeric or alphabetical, ascending or descending.

The NumPy ndarray object has a function called **sort()**, that will sort a specified array.

In [64]:
```
import numpy as np

arr = np.array([3, 2, 0, 1])

print(np.sort(arr))
```

```
[0 1 2 3]
```

In [65]:
```
import numpy as np

arr = np.array(['banana', 'cherry', 'apple'])

print(np.sort(arr))
```

```
['apple' 'banana' 'cherry']
```

In [66]:
```
import numpy as np

arr = np.array([True, False, True])

print(np.sort(arr))
```

```
[False  True  True]
```

In [67]:
```
import numpy as np

arr = np.array([[3, 2, 4], [5, 0, 1]])

print(np.sort(arr))
```

```
[[2 3 4]
 [0 1 5]]
```

Random numbers using numpy

Generate a random integer from 0 to 100:

In [68]:
```
from numpy import random
x=random.randint(100)
print(x)
```

```
93
```

In [69]:
```
#Generate a random float from 0 to 1:
y = random.rand()
print(y)
```

```
0.47561595580422733
```

Generate a 1-D array containing 5 random integers from 0 to 100:

In [71]:
```
from numpy import random

z=random.randint(100, size=(5))

print(z)
```

```
[67 27 12 51 86]
```

Generate a 2-D array with 3 rows, each row containing 5 random integers from 0 to 100:

In [72]:
```
from numpy import random

x = random.randint(100, size=(3, 5))

print(x)
```

```
[[29 55 13 88 38]
 [61 46 65 59 24]
 [17  6 66 49 50]]
```

In [73]:
```
# float
from numpy import random

x = random.rand(5)

print(x)
```

```
[0.00965153 0.18347566 0.30807387 0.58249904 0.1865615 ]
```

In [74]:
```
from numpy import random

x = random.rand(3, 5)

print(x)
```

```
[[0.60840718 0.4623859  0.94445205 0.04095272 0.80683867]
 [0.67478009 0.29308843 0.44266936 0.79958652 0.79336238]
 [0.4595015  0.32497208 0.98072312 0.18161971 0.12375534]]
```

Generate Random Number From Array The **choice()** method allows you to generate a random value based on an array of values.

The choice() method takes an array as a parameter and randomly returns one of the values.

In [75]:
```
from numpy import random

x = random.choice([3, 5, 7, 9])

print(x)
```

```
3
```

In [76]:
```
#Generate a 2-D array that consists of the values in the array parameter (3, 5, 7, and 9):
from numpy import random

x = random.choice([3, 5, 7, 9], size=(3, 5))

print(x)
```

```
[[5 7 7 9 3]
 [7 5 9 3 9]
 [7 3 7 3 7]]
```

Statistics

In [77]:
```
a = np.array([[7, 2], [5, 4]])
print(np.mean(a))
print(np.median(a))
print(np.std(a))
print(np.var(a))
print(np.min(a))
print(np.max(a))
print(np.average(a))
```

```
4.5
4.5
1.8027756377319946
3.25
2
7
4.5
```

In [78]:
```
# to cumpute covariance and correlation of two matrices
import numpy as np
x = np.array([0, 1, 2])
y = np.array([2, 1, 0])
print("\nOriginal array1:")
print(x)
print("\nOriginal array1:")
print(y)
print("\nCovariance matrix of the said arrays:\n",np.cov(x, y))
print("\nCorrelation of the said arrays:\n",np.correlate(x,y))
```

```
Original array1:
[0 1 2]

Original array1:
[2 1 0]

Covariance matrix of the said arrays:
 [[ 1. -1.]
 [-1.  1.]]

Correlation of the said arrays:
 [1]
```

Linear Algebra

In [79]:
```
# to compute determinant of square matrix
import numpy as np
from numpy import linalg as LA
a = np.array([[1, 1], [1, 2]])
print("Original 2-d array")
print(a)
print("Determinant of the said 2-D array:")
print(np.linalg.det(a))
```

```
Original 2-d array
[[1 1]
 [1 2]]
Determinant of the said 2-D array:
1.0
```

In [80]:
```
# how to print eigen values and eigen vectors
# np.mat can also be used to generate matrix
import numpy as np
m = np.mat("3 -2;1 0")
print("Original matrix:")
print("a\n", m)
w, v = np.linalg.eig(m)
print( "Eigenvalues of the said matrix",w)
print( "Eigenvectors of the said matrix",v)
```

```
Original matrix:
a
 [[ 3 -2]
 [ 1  0]]
Eigenvalues of the said matrix [2. 1.]
Eigenvectors of the said matrix [[0.89442719 0.70710678]
 [0.4472136  0.70710678]]
```

In [81]:
```
# calculating inverse of a matrix
# Python program to inverse
# a matrix using numpy
# Import required package
import numpy as np
# Taking a 3 * 3 matrix
A = np.array([[6, 1, 1],
[4, -2, 5],
[2, 8, 7]])
# Calculating the inverse of the matrix
print(np.linalg.inv(A))
```

```
[[ 0.17647059 -0.00326797 -0.02287582]
 [ 0.05882353 -0.13071895  0.08496732]
 [-0.11764706  0.1503268   0.05228758]]
```

Numpy Array Arthimetics

In [82]:
```
#BASIC ARITHMETIC
a = np.array([10,32,12])
b = np.array([3,4,5])
print(a+b)
# or
print(np.add(a,b))
print(a-b)
# or
print(np.subtract(a,b))
print(a*b)
# or
print(np.multiply(a,b))
print(a/b)
# or
print(np.divide(a,b))
print(np.mod(a,b)) # it returns remainder after dividing a by b
# or
print(np.remainder(a, b))
print(np.divmod(a, b)) #it returns quotient and remainder both
```

```
[13 36 17]
[13 36 17]
[ 7 28  7]
[ 7 28  7]
[ 30 128  60]
[ 30 128  60]
[3.33333333 8.         2.4       ]
[3.33333333 8.         2.4       ]
[1 0 2]
[1 0 2]
(array([3, 8, 2], dtype=int32), array([1, 0, 2], dtype=int32))
```

In [87]:
```
#Summation

import numpy as np

arr1 = np.array([1, 1, 2])
arr2 = np.array([1, 3, 2])

total_sum = np.sum([arr1,arr2])           # overall sum of all elements in array1 and array2
print(total_sum)

first_arr = np.sum([arr1, arr2], axis=1)    # if axis = 0 means sum across each indidual array
second_arr = np.sum([arr1, arr2], axis=0)   # if axis =1 means adding first element of both array the
print(first_arr)
print(second_arr)
```

```
10
[4 6]
[2 4 4]
```

In [88]:
```
#Product

import numpy as np

arr1 = np.array([1, 1, 2])
arr2 = np.array([1, 3, 1])

total_prod = np.product([arr1,arr2])           # overall product of all elements in array1 and array2
print(total_prod)

first_arr = np.product([arr1, arr2], axis=1)    # if axis = 0 means product across each indidual array
second_arr = np.product([arr1, arr2], axis=0)   # if axis =1 means multiply first element of both arr
print(first_arr)
print(second_arr)
```

```
6
[2 3]
[1 3 2]
```

In [85]:
```
# lCM (least common multiple)
import numpy as np
num1 = 4
num2 = 6
x = np.lcm(num1, num2)
print(x)
print("------------------------------")
# LCM of array
import numpy as np
arr = np.array([3, 6, 9])
x = np.lcm.reduce(arr)
print(x)
```

```
12
------------------------------
18
```

In [86]:
```
# Find GCD(greatest common divisor) or HCF(Highest common factor) using numpy arrays
import numpy as np
num1 = 6
num2 = 9
x = np.gcd(num1, num2)
print(x)
print("------------------------------")
import numpy as np
arr = np.array([20, 8, 32, 36, 16])
x = np.gcd.reduce(arr)
print(x)
```

```
3
------------------------------
4
```

Set operations

Convert following array with repeated elements to a set:

In [89]:
```
import numpy as np

arr = np.array([1, 1, 1, 2, 3, 4, 5, 5, 6, 7])

x = np.unique(arr)

print(x)
```

```
[1 2 3 4 5 6 7]
```

Find union of the following two set arrays:

In [90]:
```
import numpy as np

arr1 = np.array([1, 2, 3, 4])
arr2 = np.array([3, 4, 5, 6])

newarr = np.union1d(arr1, arr2)

print(newarr)
```

```
[1 2 3 4 5 6]
```

Find intersection of the following two set arrays:

In [91]:
```
import numpy as np

arr1 = np.array([1, 2, 3, 4])
arr2 = np.array([3, 4, 5, 6])

newarr = np.intersect1d(arr1, arr2, assume_unique=True)

print(newarr)
```

```
[3 4]
```

Find the difference of the set1 from set2:

In [92]:
```
import numpy as np

set1 = np.array([1, 2, 3, 4])
set2 = np.array([3, 4, 5, 6])

newarr = np.setdiff1d(set1, set2, assume_unique=True)

print(newarr)
```

```
[1 2]
```

Trignometric

In [93]:
```
import numpy as np

x = np.sin(np.pi/2)

print(x)
```

```
1.0
```

Convert all of the values in following array arr to radians:

In [94]:
```
import numpy as np

arr = np.array([90, 180, 270, 360])

x = np.deg2rad(arr)

print(x)
```

```
[1.57079633 3.14159265 4.71238898 6.28318531]
```

Radians to Degrees

In [95]:
```
import numpy as np

arr = np.array([np.pi/2, np.pi, 1.5*np.pi, 2*np.pi])

x = np.rad2deg(arr)

print(x)
```

```
[ 90. 180. 270. 360.]
```

NumPy provides ufuncs arcsin(), arccos() and arctan() that produce radian values for corresponding sin, cos and tan values given.

In [96]:
```
import numpy as np

x = np.arcsin(1.0)

print(x)
```

```
1.5707963267948966
```

In [97]:
```
import numpy as np

arr = np.array([1, -1, 0.1])

x = np.arcsin(arr)

print(x)
```

```
[ 1.57079633 -1.57079633  0.10016742]
```

In [98]:
```
#NumPy provides the hypot() function that takes the base and perpendicular values
#and produces hypotenues based on pythagoras theorem.
import numpy as np

base = 3
perp = 4

x = np.hypot(base, perp)

print(x)
```

```
5.0
```

Numpy where and choice

In [4]:
```
import numpy as np
x=np.array([i for i in range(10)])
print(x)
```

```
[0 1 2 3 4 5 6 7 8 9]
```

In [5]:
```
np.where(x%2==0,'Even','Odd')
```

Out[5]:
```
array(['Even', 'Odd', 'Even', 'Odd', 'Even', 'Odd', 'Even', 'Odd', 'Even',
       'Odd'], dtype='<U4')
```

In [6]:
```
condlist=[x>5,x<5]
choicelist=[x**2,x**3]
```

In [7]:
```
np.select(condlist,choicelist,default=x)
```

Out[7]:
```
array([ 0,  1,  8, 27, 64,  5, 36, 49, 64, 81])
```

In [17]:
```
# axis=0 for vertical,axis=1 for horizontal
y=np.arange(10,22)
print(y)
```

```
[10 11 12 13 14 15 16 17 18 19 20 21]
```

In [18]:
```
y.reshape(2,6)
```

Out[18]:
```
array([[10, 11, 12, 13, 14, 15],
       [16, 17, 18, 19, 20, 21]])
```

In [20]:
```
np.amin(y,axis=0)
```

Out[20]:
```
10
```

In [ ]:

In [ ]: